

ITR Lab 3: Pick and Place in MuJoCo with Inverse Kinematics

Franka Emika Panda and HEAL — Mink vs. DLS-IK vs. QP-IK

Parshva Patel (Roll No: 24110240) | Repository: [LINK](#)

1 Introduction and goals

A cube is spawned at a random pose (x, y, ψ) on a table and must be picked and placed into a tray, for two manipulators: the **Franka Emika Panda** (7-DoF, Franka Hand) and the **HEAL** arm (6-DoF, Robotiq 2F-85). The same pipeline is run with three inverse-kinematics (IK) solvers: the off-the-shelf **Mink** library (<https://github.com/kevinzakka/mink>), a closed-loop **Damped Least Squares** (DLS) solver, and a **Quadratic-Programming** (QP) solver with joint and velocity limits. For each robot we report the workspace analysis, the table design derived from it, the logged batch results, the failure cases, and a final comparison of the three IK methods on identical episode seeds.

2 Methods

Environment. The scene is built with MuJoCo’s `MjSpec`: robot and gripper, a `tcp` site between the finger pads (z = approach direction, x = closing axis, identical for both robots), a box table, an open tray (floor + four walls, 14×14 cm inner, 5 cm high) and a 4 cm, 50 g free-floating cube. Grasp stability required explicit cube inertia, torsional/rolling friction (`condim=6`), and collision masks so the cube contacts only the finger pads while the finger hulls still detect table/tray hits.

Randomisation. Episode i uses seed $s_0 + i$; (x, y, ψ) are drawn uniformly from the spawn box (plus a radial band and a 6 cm tray keep-out) by rejection sampling, so every IK method sees identical cubes.

Control. IK produces joint targets $\mathbf{q}_{cmd}$. HEAL is torque-driven: $\boldsymbol{\tau} = \mathbf{b}(\mathbf{q}, \dot{\mathbf{q}}) + K_p(\mathbf{q}_{cmd} - \mathbf{q}) + K_d(\dot{\mathbf{q}}_{cmd} - \dot{\mathbf{q}}) + D\dot{\mathbf{q}}_{cmd}$ (gravity/Coriolis compensation `qfric_bias`, PD, joint-damping feed-forward); the Panda uses its built-in joint position servos. Grippers are commanded through their actuators.

IK methods (common interface, pose error $\mathbf{e} = [\Delta\mathbf{p}; \varepsilon]$, $\varepsilon = 2 \text{Log}(q_t \otimes q_c^{-1})$ with hemisphere flip; tolerance 2 mm / 0.02 rad, ≤ 150 iterations, $\dot{\mathbf{q}}_{max} = 2 \text{ rad/s}$):

- **DLS:** $\dot{\mathbf{q}} = (J^\top J + \lambda I)^{-1} J^\top K \mathbf{e} + (I - (J^\top J + \lambda I)^{-1} J^\top J) \alpha(\mathbf{q}_{mid} - \mathbf{q})$, $\lambda = 10^{-3}$; $\dot{\mathbf{q}}$ is scaled to the velocity limit, joint limits are not enforced.
- **QP:** $\min \frac{1}{2} \Delta \mathbf{q}^\top H \Delta \mathbf{q} + c^\top \Delta \mathbf{q}$, $H = \frac{2}{\Delta t^2} (J^\top J + \lambda I)$, $c = -\frac{2}{\Delta t} J^\top \dot{\mathbf{x}}_d$, s.t. $[I; -I; I; -I] \Delta \mathbf{q} \leq [\mathbf{q}_{max} - \mathbf{q}; \mathbf{q} - \mathbf{q}_{min}; \dot{\mathbf{q}}_{max} \Delta t; \dot{\mathbf{q}}_{max} \Delta t]$ (DAQP solver).
- **Mink:** `FrameTask` on the `TCP` + `ConfigurationLimit` + `VelocityLimit`; gripper DoFs frozen with `DofFreezingTask`.

Motion plan and cube–tray collision avoidance. Waypoints: pre-grasp (12 cm above) $\rightarrow$ grasp $\rightarrow$ lift $\rightarrow$ above-tray $\rightarrow$ place $\rightarrow$ release $\rightarrow$ retreat. All waypoints are IK-checked before moving; the four symmetric grasp yaws $\psi + k\pi/2$ are tried (closest to the approach direction first). Segments are straight Cartesian lines (smoothstep timing, quaternion slerp) with IK streamed at 50 Hz. To avoid the tray: (i) the transit height is computed from the *held cube*, so its bottom clears the wall top by 4 cm; (ii) the cube is carried to directly above the tray centre and lowered vertically; (iii) the place yaw is snapped to the tray axes; (iv) the rotated footprint (+5 mm) is checked against the inner tray.

Monitoring and success. Every physics step checks contacts (robot–table, robot–tray, cube–wall before release, arm–cube), grasp slip (>3 cm), and tracking error (>3 cm); clearances are measured with `mj_geomDistance`. Success = cube at rest inside the tray. Each failure is logged with its reason, phase and a screenshot. Per-episode logs (`episodes.csv`) store the initial pose, success, failure reason, minimum clearances, planning time, IK time and iterations, and joint-limit/velocity violations. Three experiments per robot and method: **nominal** (25 episodes, seeds 0–24, designed spawn region), **stress** (40 episodes, seeds 100–139, whole table) and a **no-avoidance ablation** (nominal seeds, diagonal approach into the tray, no yaw snapping).

3 Franka Emika Panda

3.1 Workspace analysis

The *task workspace* (Fig. 1a) is obtained from 50 000 random joint configurations inside the joint limits (forward kinematics of the TCP): the Panda reaches $r \leq 0.95$ m. The *dexterous workspace* (Fig. 1b) is computed on a 4 cm grid at the grasp height ($z = 0.12$ m) and the transit height ($z = 0.25$ m): for each cell, DLS-IK attempts a top-down TCP at 8 yaws in $[-\pi, \pi)$ within joint limits. Two features stand out: no top-down pose exists close to the base ($r < 0.25$ m, red), and most of the useful area reaches 75–90 % of yaws rather than all of them, because of the joint-7 limit (± 2.9 rad).

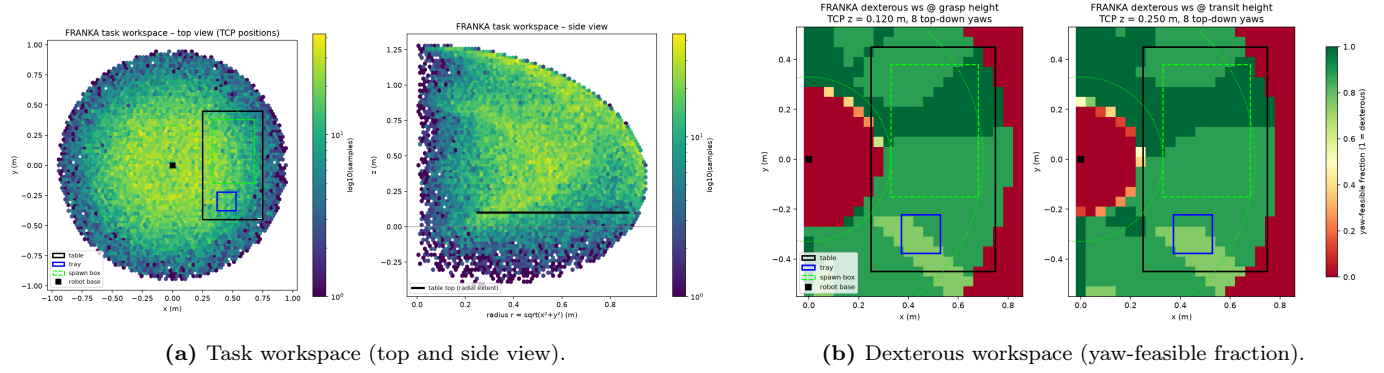


Figure 1: Franka workspace with the chosen table (black), tray (blue) and spawn region (green dashed).

3.2 Table design

Parameter	Symbol	Value	Rationale
Table length	L	0.50 m ($x \in [0.25, 0.75]$)	Starts outside the unreachable zone near the base ($r < 0.25$ m); ends near the dexterous limit ($r \approx 0.75$ m).
Table width	W	0.90 m ($y \in [-0.45, 0.45]$)	Holds spawn region + tray; outer corners deliberately exceed reach (used by the stress test).
Table height	H	0.10 m	Top-down poses exist down to table level over the whole band, so a low table keeps grasp ($z = 0.12$) and transit ($z = 0.25$) heights inside the dexterous region without joint limits.
Spawn region	–	$x \in [0.33, 0.68]$, $y \in [-0.15, 0.38]$, $r \in [0.33, 0.70]$	Mean yaw-feasibility 0.92 (min 0.88) at grasp height.
Tray	–	centre (0.45, –0.30)	Mean yaw-feasibility 0.80 (min 0.75); outside the spawn region.

Table 1: Franka table design (robot base at the origin).

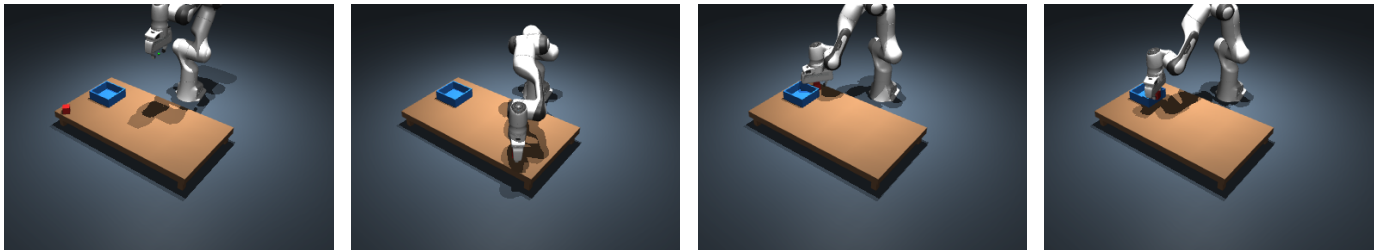
3.3 Logs and summary metrics

Experiment	Method	Success	Plan time	IK time /call	Plan IK it.	Max it./call	Min clr. cube–tray	Track err.	Vel. viol. steps
Nominal (25)	Mink	25/25	6.58 ms	0.149 ms	66.0	9	4.41 cm	8.6 mm	319
	DLS	25/25	4.22 ms	0.084 ms	77.2	1	4.30 cm	8.6 mm	0
	QP	25/25	4.59 ms	0.098 ms	66.3	8	4.41 cm	8.6 mm	321
Stress (40)	Mink	29/40	8.65 ms	0.156 ms	84.9	20	0.77 cm	9.2 mm	232
	DLS	29/40	4.77 ms	0.082 ms	88.6	1	0.77 cm	9.3 mm	0
	QP	29/40	5.90 ms	0.097 ms	84.4	20	0.77 cm	9.3 mm	228
No avoidance (25)	all	0/25	–	–	–	–	0.03 cm	–	–

Table 2: Franka summary (from `logs/franka/*/summary.json`). Joint-limit violations: 0 in every run. Velocity violations = simulation steps with $|\dot{q}_i| > 2$ rad/s.

All three methods succeeded in 25/25 nominal episodes (mean episode duration 13.3 s simulated). The **stress** test (whole table) gives 29/40 for every method, with identical failures on identical seeds. Without the cube–tray avoidance, 0/25 succeed (22× cube hits a tray wall, 3× gripper hits the tray): minimum clearance drops from 4.3 cm to 0.03 cm, which validates the avoidance strategy (Fig. 5).

3.4 Failure cases



(a) `ik_infeasible` (b) `collision_table` (c) `collision_tray` (d) `collision_cube_tray`

Figure 2: Franka failure snapshots (taken at the failure instant): (a)–(c) stress test, (d) no-avoidance ablation.

Failure	Count	Cause	Mitigation
collision_table	5/40	All at $r > 0.72$ m: at full extension the position servos sag (no gravity feed-forward) and the finger hull touches the table during the descent.	Keep spawn $r \leq 0.70$ m (done); add gravity compensation or a few mm of grasp height.
collision_tray	5/40	Cube spawned next to the tray; the hand body contacts a tray wall while descending.	Tray keep-out in spawn (done); choose a grasp yaw that keeps the fingers parallel to the wall.
ik_infeasible	1/40	Cube at $r = 0.81$ m, pre-grasp outside the reachable region.	Spawn radial band (done); tilted approach.
collision_cube_tray	22/25	Ablation only: diagonal path drags the cube through the wall.	Transit height from held cube + above-then-down.

Table 3: Franka failure categories (stress test unless noted).

4 HEAL

4.1 Workspace analysis

The same analysis was run on the HEAL model (Fig. 3). HEAL reaches $r \leq 0.85$ m (task workspace). Its dexterous workspace is very different from the Panda’s: close to the base height top-down poses only exist in a thin ring ($r \approx 0.45$ – 0.62 m), whereas at $z = 0.20$ – 0.36 m *every* yaw is feasible over the whole disc $r \lesssim 0.55$ m (joint ranges are $\pm\pi$). The dexterous region ends at $x \approx 0.55$ – 0.6 m.

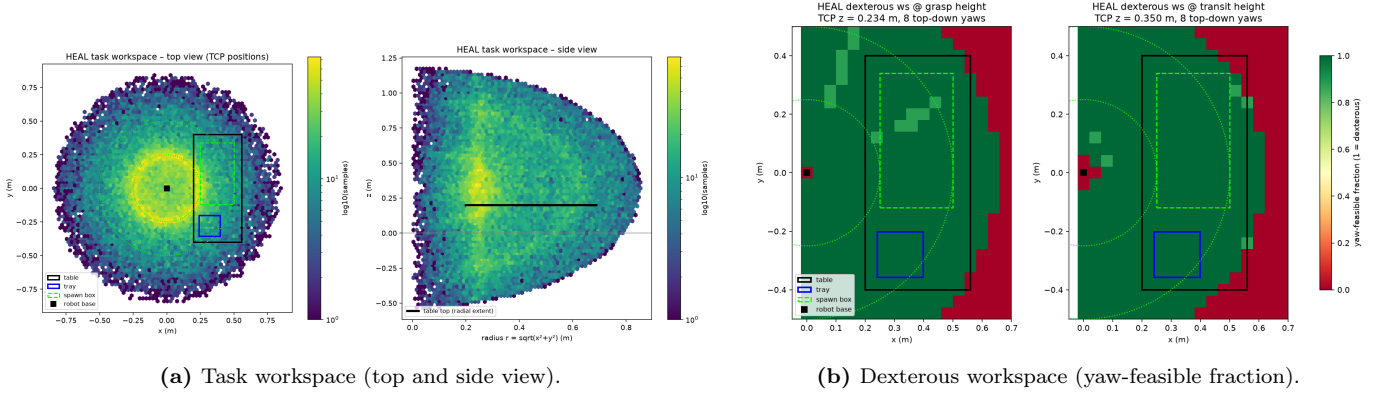


Figure 3: HEAL workspace with the chosen table, tray and spawn region.

4.2 Table design

Parameter	Symbol	Value	Rationale
Table length	L	0.36 m ($x \in [0.20, 0.56]$)	Ends at the dexterous limit ($x \approx 0.55$ m); starts clear of the base.
Table width	W	0.80 m ($y \in [-0.40, 0.40]$)	Spawn region + tray; corners exceed the reach band (stress test).
Table height	H	0.20 m	Raising the table puts the grasp height ($z = 0.234$ m) and transit height ($z = 0.35$ m) in the fully dexterous band; at lower heights only a thin ring is feasible.
Spawn region	—	$x \in [0.25, 0.50]$, $y \in [-0.12, 0.34]$, $r \in [0.25, 0.50]$	Mean yaw-feasibility 0.99 (94 % of cells fully dexterous).
Tray	—	centre (0.32, -0.28)	Yaw-feasibility 1.00 in every cell.

Table 4: HEAL table design.

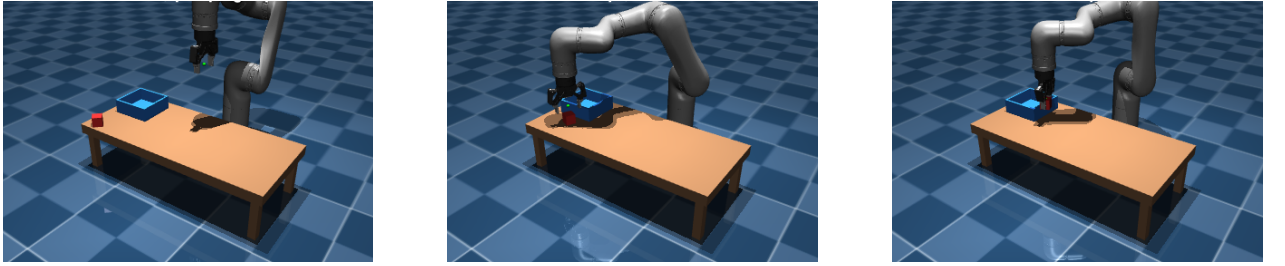
4.3 Logs and summary metrics

Experiment	Method	Success	Plan time	IK time /call	Plan IK it.	Max it./call	Min clr. cube-tray	Track err.	Vel. viol. steps
Nominal (25)	Mink	25/25	9.77 ms	0.158 ms	97.3	1	3.24 cm	1.8 mm	0
	DLS	25/25	4.91 ms	0.086 ms	93.0	1	3.23 cm	1.8 mm	0
	QP	25/25	6.75 ms	0.102 ms	97.5	1	3.23 cm	1.8 mm	0
Stress (40)	Mink	37/40	10.90 ms	0.157 ms	107.9	1	1.16 cm	1.8 mm	0
	DLS	37/40	5.64 ms	0.085 ms	108.0	1	1.16 cm	1.8 mm	0
	QP	37/40	7.74 ms	0.103 ms	108.5	1	1.16 cm	1.8 mm	0
No avoidance (25)	all	0/25	—	—	—	—	0.03 cm	—	—

Table 5: HEAL summary (from `logs/heal/*/summary.json`). Joint-limit violations: 0 in every run.

All methods succeed in 25/25 nominal episodes (11.9s per episode) and 37/40 in the stress test. Tracking is much tighter than on the Panda (1.8 vs 8.6 mm) because HEAL is torque-controlled with gravity compensation. The ablation again drops to 0/25 ($21\times$ cube-wall, $4\times$ gripper-tray).

4.4 Failure cases



(a) `ik_infeasible`

(b) `collision_tray`

(c) `collision_cube_tray`

Figure 4: HEAL failure snapshots: (a)–(b) stress test, (c) no-avoidance ablation.

- **ik_infeasible (1/40):** cube at $(0.525, -0.340)$, $r = 0.62$ m – outside the dexterous band; mitigated by the radial spawn band (or a tilted approach).
- **collision_tray (2/40):** cube spawned beside the tray; the 2F-85 fingers open over the wall during the descent. Mitigation: tray keep-out (used in the nominal set) or a grasp yaw parallel to the wall.
- **Development failures** (fixed before the reported runs): cube launched at spawn (wrong inertia $\rightarrow$ explicit inertia), fingertips striking the table while closing (2F-85 arc $\rightarrow$ grasp 14 mm above the cube centre), cube squeezed out by convex finger hulls ($\rightarrow$ collision masks), and `grasp_slip` during the place descent (8/16 $\rightarrow$ 16/16 with `condim=6` and a 25 N grip limit).

5 Comparison of the three IK methods

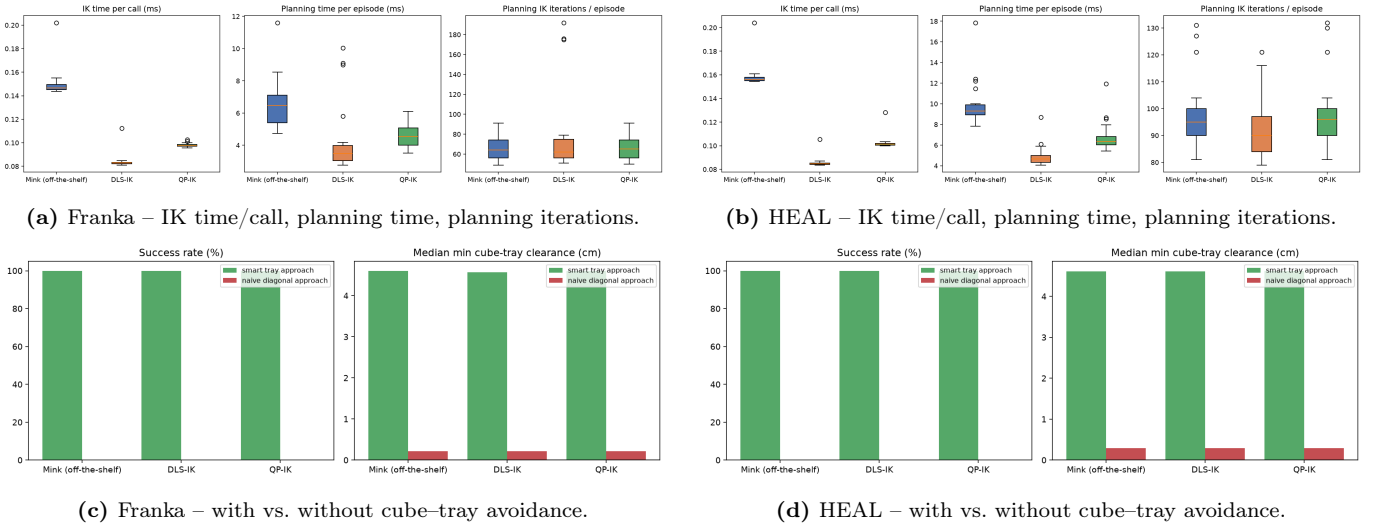


Figure 5: IK comparison on identical seeds (nominal set) and the collision-avoidance ablation.

	Mink (off-the-shelf)	DLS-IK	QP-IK
Success (nominal / stress)	100 % / 72.5 % (Franka), 100 % / 92.5 % (HEAL)	identical	identical
Planning time vs. DLS	1.6× (Franka), 2.0× (HEAL)	fastest (4.2 / 4.9 ms)	+9 % (Franka), +37 % (HEAL)
IK time per call	0.15–0.16 ms	0.08–0.09 ms	0.10 ms
Planning iterations (Franka / HEAL)	66.0 / 97.3	77.2 / 93.0	66.3 / 97.5
Joint-limit violations	0 (hard constraint)	0 (not enforced, never reached)	0 (hard constraint)
Velocity-limit steps (Franka)	319	0	321
Strengths	Fast to integrate; general task/limit framework	Simplest, fastest, robust near singularities; null space keeps joints centred	Joint and velocity limits as hard constraints; same cost as DLS
Weaknesses	Less control over internals; overhead	Limits only implicitly (scaling / null space); λ tuning	Needs a QP solver; slightly slower

Table 6: Final comparison of the three IK methods (identical seeds, both robots).

Discussion.

- **Same outcomes, different cost.** On identical seeds the three solvers produce exactly the same successes and failures on both robots: all of them find valid solutions wherever one exists, so failures are caused by reachability and geometry, not by the IK method. The methods differ in computational cost: closed-form DLS (one $n \times n$ linear solve per iteration) is fastest, QP adds a solver call, and Mink is 1.6–2× slower than DLS because of its general task and limit framework.
- **Streaming vs. planning.** Streaming at 50 Hz moves the target only millimetres per tick, so most calls converge in ≤ 1 iteration; differences appear in planning (large jumps from home). On the redundant Panda, Mink and QP need fewer planning iterations (66) than DLS (77), because the DLS null-space term (joint centring) competes slightly with convergence.
- **Velocity limits.** On the Panda, Mink and QP produced ≈ 320 simulation steps above 2 rad/s while DLS produced none. Both enforce the limit per *iteration*, but on a few wrist-reorientation ticks they ran up to 8–9 iterations inside one 50 Hz tick (max iterations/call), so the per-tick joint motion exceeded the bound. DLS (1 iteration/call, null-space centring) yields a smoother wrist path. Fix: bound the total $\Delta \mathbf{q}$ per tick or run one QP step per tick.
- **Joint limits.** HEAL’s joints are $\pm\pi$, so limits never become active there. On the Panda the alternate-grasp-yaw search discards yaws whose waypoint chain would leave the joint range, so no method violated a joint limit.
- **Robot comparison.** The Panda covers a larger table (0.50×0.90 m at $H = 0.10$ m) but is only partially yaw-dexterous (joint-7 limit), and its position servos sag at full reach, which causes its extra stress failures (72.5 % vs 92.5 %). HEAL is fully yaw-dexterous within its smaller reach and tracks 5× more accurately thanks to torque control with gravity compensation.

6 Conclusions

A complete randomised pick-and-place pipeline was implemented in MuJoCo for the Franka Panda and HEAL, with three interchangeable IK methods. Table dimensions were derived from task and dexterous workspace analysis, and the cube-tray avoidance strategy is essential (100 % vs 0 % success). In the designed workspace all methods reach 100 % success; in the whole-table stress test the remaining failures are geometric (reach limits and tray proximity). DLS was the fastest and smoothest solver here, QP offers hard constraint handling at a small cost, and Mink is the quickest to integrate. Future work: gravity compensation for the Panda servos, wall-aware grasp-yaw selection, and one-step-per-tick QP execution to respect velocity limits strictly.